

Пречки за лама

Лама сака да патува низ Андската Висорамнина. Таа има мапа на висорамнината во форма на мрежа од $N \times M$ квадратни клетки. Редовите на мапата се нумерирани од 0 до $N - 1$ од горе надолу, а колоните се нумерирани од 0 до $M - 1$ од лево кон десно. Клетката на мапата во редот i и колоната j ($0 \leq i < N, 0 \leq j < M$) е означена со (i, j) .

Ламата ја проучила климата на висорамнината и открила дека сите клетки во секој ред од мапата имаат иста **температура** и сите клетки во секоја колона од мапата имаат иста **влажност**. Ламата ви дала две целобројни низи T и H со должина N и M соодветно. Тука $T[i]$ ($0 \leq i < N$) ја означува температурата на клетките во редот i , а $H[j]$ ($0 \leq j < M$) ја означува влажноста на клетките во колоната j .

Ламата, исто така, ја проучила флората на висорамнината и забележала дека клетката (i, j) е **без вегетација** ако и само ако нејзината температура е поголема од влажноста, формално $T[i] > H[j]$.

Ламата може да патува низ висорамнината само следејќи **валидни патеки**. Валидна патека е низа од различни клетки кои ги задоволуваат следниве услови:

- Секој пар последователни клетки во патеката дели заедничка страна.
- Сите клетки на патеката се без вегетација.

Вашата задача е да одговорите на Q прашанки. За секоја прашанка ви се дадени четири цели броеви: L, R, S и D . Вие мора да утврдите дали постои валидна патека таква што:

- Патеката започнува во клетката $(0, S)$ и завршува во клетката $(0, D)$.
- Сите клетки во патеката се наоѓаат во рамките на колоните L до R , вклучително.

Гарантирано е дека и $(0, S)$ и $(0, D)$ се без вегетација.

Детали за имплементација

Првата процедура што треба да ја имплементирате е:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : низа со должина N што ја специфицира температурата во секој ред.
- H : низа со должина M што ја специфицира влажноста во секоја колона.

- Оваа процедура се повикува точно еднаш за секој тест случај, пред какви било повици на `can_reach`.

Втората процедура што треба да ја имплементирате е:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : цели броеви што опишуваат прашанка.
- Оваа процедура се повикува Q пати за секој тест случај.

Оваа процедура треба да врати `true` ако и само ако постои валидна патека од клетката $(0, S)$ до клетката $(0, D)$, така што сите клетки на патеката лежат во рамките на колоните L до R , вклучително.

Ограничувања

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ за секој i таков што $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ за секој j таков што $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- И двете клетки $(0, S)$ и $(0, D)$ се без вегетација.

Подзадачи

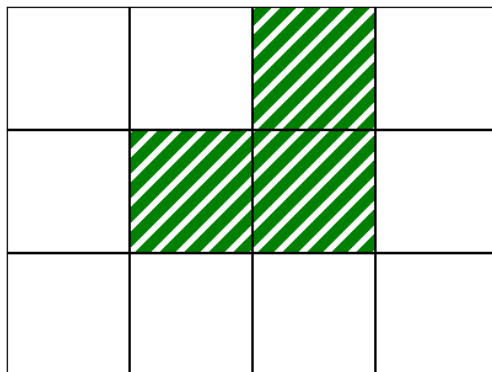
Подзадача	Поени	Дополнителни ограничувања
1	10	$L = 0, R = M - 1$ за секоја прашанка. $N = 1$.
2	14	$L = 0, R = M - 1$ за секоја прашанка. $T[i - 1] \leq T[i]$ за секое i така што $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ за секоја прашанка. $N = 3$ и $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ за секоја прашанка. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ за секоја прашанка.
6	17	Без дополнителни ограничувања.

Пример

Да го разгледаме следниов повик:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

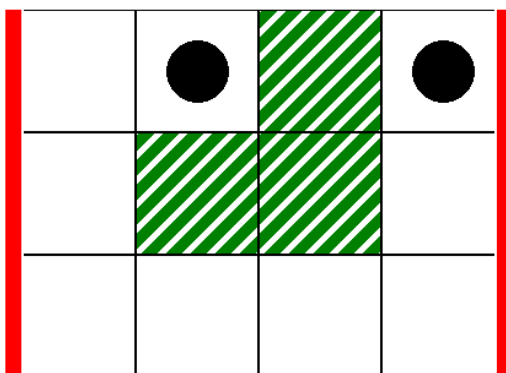
Ова одговара на мапата на следната слика, каде што белите клетки се без вегетација:



Како прва прашанка, разгледајте го следниов повик:

```
can_reach(0, 3, 1, 3)
```

Ова одговара на сценариото на следната слика, каде што дебелите вертикални линии го означуваат опсегот на колони од $L = 0$ до $R = 3$, а црните дискови ги означуваат почетните и крајните клетки:



Во овој случај, ламата може да стигне од клетката $(0, 1)$ до клетката $(0, 3)$ преку следната валидна патека:

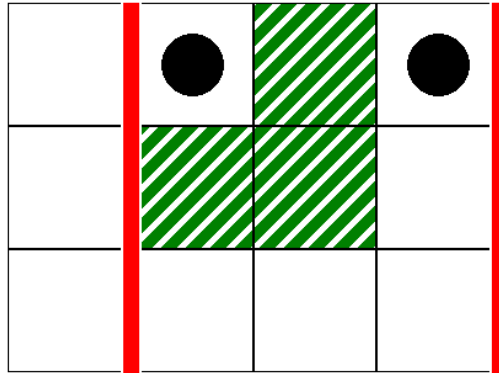
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Затоа, овој повик треба да врати `true`.

Како втора прашанка, разгледајте го следниов повик:

```
can_reach(1, 3, 1, 3)
```

Ова одговара на сценариото на следната слика:



Во овој случај, не постои валидна патека од клетката $(0, 1)$ до клетката $(0, 3)$, така што сите клетки во патеката лежат во рамките на колоните 1 до 3, вклучително. Затоа, овој повик треба да врати `false`.

Пример оценувач

Влезен формат:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Тука, $L[k], R[k], S[k]$ и $D[k]$ ($0 \leq k < Q$) ги одредуваат параметрите за секој повик на `can_reach`.

Излезен формат:

```
A[0]
A[1]
...
A[Q-1]
```

Тука, $A[k]$ ($0 \leq k < Q$) е 1 ако повикот `can_reach(L[k], R[k], S[k], D[k])` врати `true`, а во спротивно 0.